Week 9 · Making it look right: visual design & accessibility

Technical Interaction Design · ITU · Autumn 2026

Built 4 September 2026 · commit `4e11e8b`
Latest version: https://itu-tid.github.io/lecture-notes-pdf/Week-09.pdf

CONTENTS

Libraries

Source: Lectures/Technical/Tooling/Libraries.md — something unclear? Open an issue, or send a pull request.

“If I have seen further, it is by standing on the shoulders of giants.” (Isaac Newton)

(the programmers’ version of the same idea)

Modern programming languages allow you to specify a list of required libraries for every project

- In React
  - you do this by adding all the libraries on which you depend on to the `package.js`
  - Under the `dependencies` key

React is just a library that runs on `node` – a runtime environment for executing JavaScript outside the browser

- If you look in our ToDo list your project you’ll see that React is also in the `dependencies` list
- Node uses the same JavaScript engine as Chrome (V8), but provides additional APIs and features designed for server-side and command-line applications.

This is supported by `package managers`

- every programming language has its own
- in JS the package manager is `npm`
- When you clone a project, you always run `npm install` to install the required libraries

One of the most popular version numbering schemes is semantic versioning

- a version is specified by three
  - major
  - minor
  - patch
- other version numbering schemes?
  - calendaristic: pip
  - idiosyncratic: latex

Libraries are normally specified with their version number and pinned or flexible

- pinned = exactly a given version
- flexible = allows the package manager to retrieve newer minor and patch versions automatically
- however, even if you pin all the dependencies, the transitive dependencies might still change

Most package managers differentiate between a high-level dependency list and a `lockfile`

React creates a `pacakage.lock` file to pin exactly the versions of all the package dependency tree

- this one you don't commit to the repo normally - it's generated
- unless you want to ensure 100% replicability of all the dependencies

Should you push `lockfile`

- yes – to ensure perfect replicability of the code
- no – because it's generated and we don't normally put generated code under version control
- yes – it's not exactly generated - it's configuration

The libraries you install are saved in a local folder

- in node this is `node_modules`
- that one you NEVER want to commit to git!

Questions to be able to answer

- How do you instal dependencies with `npm`? What do you do to make sure that your colleagues also use the same dependency that you are using?
- When do you have to run `nmp install`?
- What is the `node_modules` folder good for? Should you add it to version control? Why or why not?
- Why do we list dependencies in `packages.json`?
- What is the meaning of the version numbers in `packages.json`?
- Why do we need `package-lock.json`? Do we commit it to GH?